

Load Balancing Basics

Phase 1 · Fundamentals · Estimated time: 1.5 hours

1. Concept From First Principles

One server can only handle so many requests before it runs out of CPU, memory, or connections. The fix is to run *multiple* copies of your application (Chapter 12 calls this horizontal scaling) — but then something needs to decide which of those servers handles each incoming request. That something is a **load balancer**: a component that sits in front of a pool of servers and distributes incoming traffic across them.

Backend engineer analogy

If you've ever run your Node/Laravel app behind Nginx with multiple PM2/PHP-FPM workers, Nginx was already acting as a mini load balancer, spreading requests across workers on the same machine. Production load balancers do the same job, just across many separate machines instead of many processes on one machine.

L4 vs L7 load balancing

Layer 4 (L4) load balancers operate at the transport layer — they route based only on IP address and port, without looking inside the actual HTTP request. They're extremely fast and simple, but "dumb": they can't route based on the URL path or headers.

Layer 7 (L7) load balancers operate at the application layer — they inspect the actual HTTP request (path, headers, cookies) and can make smarter routing decisions, like sending `/api/payments/*` to one service and `/api/search/*` to another. This flexibility costs some extra latency and CPU compared to L4. Most production systems use both: an L4 balancer at the outer edge for raw throughput and DDoS resilience, with L7 balancers or an API gateway behind it for intelligent routing.

Load balancing algorithms

- **Round Robin**: requests are distributed to servers in rotating order. Simple, works well when all servers and requests are roughly equal.
- **Least Connections**: send the next request to whichever server currently has the fewest active connections — better when request processing times vary a lot.
- **Weighted variants**: give some servers a higher share of traffic (e.g., a bigger machine gets weight 3 vs. a smaller one's weight 1).
- **IP Hash / Consistent Hashing**: route the same client consistently to the same server — useful when a server holds session state locally (a pattern this program otherwise discourages — see "stateless services" in Chapter 12).

It's worth being precise about what a load balancer does *not* solve on its own: it distributes traffic across existing capacity, but it doesn't create new capacity. If every server behind it is already maxed out, spreading requests evenly just means every server is evenly overwhelmed instead of one server being

overwhelmed while others sit idle. Load balancing and auto-scaling (Chapter 12) are complementary, not interchangeable — a common point of confusion worth pre-empting in an interview answer.

Health checks

A load balancer continuously pings each backend server (e.g., an HTTP GET on **/health**) and stops sending traffic to any server that fails to respond correctly, automatically routing around failures without human intervention. This is a core building block of high availability, covered fully in Phase 2.

One nuance worth flagging: sticky sessions (IP hash routing a client consistently to the same server) can look like a convenient shortcut when a service isn't fully stateless yet, but it quietly undermines the benefits of horizontal scaling — if that one server goes down, every client stuck to it loses their session, and load can become uneven if some "sticky" clients are far more active than others. The more durable fix, covered in Chapter 12, is making the service stateless in the first place so sticky routing isn't needed at all.

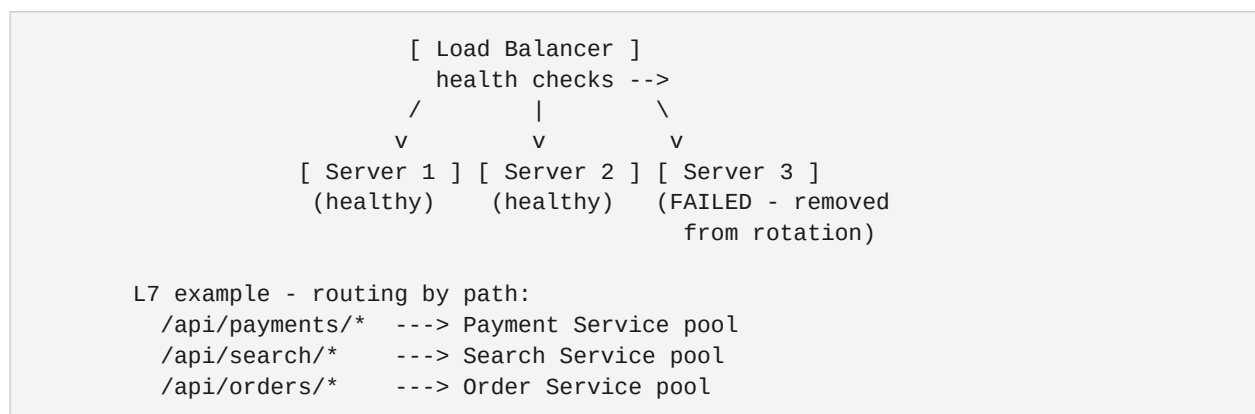
2. Real-World Example / Case Study

During a Flipkart Big Billion Days sale, traffic can spike far beyond normal levels. The load balancer layer is what lets Flipkart absorb that spike by simply adding more application server instances behind the same load balancer — without changing a single line of app code or a single client-facing URL. This is precisely why load balancers are considered foundational: they decouple "how much traffic we can handle" from "how the application is written."

Careem's API gateway layer (an L7-style component) routes ride-booking requests, payment requests, and food-delivery requests (Careem operates multiple verticals) to entirely different backend service pools based on the URL path — a textbook L7 routing decision that a simple L4 load balancer could not make.

A second example: Noon's search service, which is CPU-intensive per request, benefits particularly from least-connections load balancing rather than plain round robin — because search queries vary widely in how long they take to process (a simple keyword vs. a complex filtered query), sending the next request to whichever server is least busy right now avoids piling more work onto a server that's already struggling with a slow query, something round robin can't account for.

3. Diagram



4. Trade-offs Table

Layer / Algorithm	Strength	Weakness
L4 (transport layer)	Very fast, low overhead	Can't route on URL/headers, less flexible
L7 (application layer)	Smart routing on path/headers/cookies	More CPU/latency cost per request
Round Robin	Simple, predictable	Ignores actual server load
Least Connections	Adapts to uneven request durations	Slightly more overhead to track state
IP Hash / Consistent Hashing	Same client -> same server (sticky)	Uneven distribution if traffic pattern is skewed

5. Common Interview Questions

Q1: "Where would you place a load balancer in your architecture, and why?"

Model approach: Place it directly in front of any tier with multiple stateless instances (app servers first, but note databases can have their own load balancing too, e.g., across read replicas). Explain it removes a single point of failure and enables horizontal scaling without client-visible changes.

Q2: "How does the load balancer itself avoid becoming a single point of failure?"

Model approach: Explain that production load balancers are typically deployed in redundant pairs or clusters (active-passive or active-active) with a floating IP or DNS-based failover, so the load balancer layer itself has no single point of failure — a good moment to reference Chapter 3's DNS routing.

Q3: "Why is IP hash / sticky session routing generally discouraged in modern system design?"

Model approach: Explain it ties a client to one specific server, reintroducing the exact single-point-of-failure and uneven-load problems horizontal scaling is meant to solve. The preferred fix is making the service stateless (session data externalized to Redis or a token) so any server can handle any request, removing the need for stickiness entirely.

6. Practice Exercises

- 1 Explain, without notes, the difference between L4 and L7 load balancing to a junior engineer.
- 2 For an e-commerce app with separate search, checkout, and recommendation services, sketch an L7 routing table by URL path.
- 3 Describe what happens, step by step, when one of three backend servers behind a load balancer suddenly crashes.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Load Balancers L4 vs L7 Explained High-Level Design"	Directly covers the L4/L7 distinction this chapter centers on.

Language	Video & Channel	Why it helps
English	"Load Balancer in System Design Interviews L4 vs L7 Explained + Algorithms"	Frames the topic specifically for interview answers.
Hindi	"System Design: (P-3) Load Balancer Explained How It Works and Why You Need It"	Hindi walkthrough of core load balancer concepts.
Hindi	"Complete System Design Roadmap 2025 HLD & LLD" — CodeHelp	Covers load balancing within a broader Hindi system design roadmap.

8. Key Takeaways

- A load balancer distributes incoming traffic across multiple servers so no single one is overwhelmed.
- L4 load balancers route on IP/port only (fast, simple); L7 route on actual request content (smart, costlier).
- Common algorithms: round robin, least connections, weighted variants, IP hash/consistent hashing.
- Health checks let the load balancer automatically stop sending traffic to failed servers.
- Load balancers themselves must be deployed redundantly to avoid becoming a single point of failure.
- Load balancing decouples "how much traffic we can handle" from "how the application is written."